Binary Search and Sorting

Topic 8 of 10

17 questions | skeleton and writing space only

Each question: the prompt, the function or class signature, and ruled lines to write your own solution.

1. Check an Interval Against Existing Intervals for Conflicts

EASY | asked 2026-08-08 |

Respect strict start and end semantics so ranges that only touch at an endpoint do not conflict.

```
class Calendar:

    def __init__(self):
```

```
    def book(self, start, end):
```

2. Cutting Ribbons (LC 1891)

MEDIUM | asked 2026-08-16 | array, binary-search, complexity-analysis

LC 1891 Cutting Ribbons: given wood lengths and K, maximize the equal piece length that yields at least K pieces.

```
def max_length(ribbons, k):
```

3. First Bad Version with Parallel Search Follow-up

MEDIUM | asked 2026-06-15 | binary-search, parallelism, concurrency

Phone-screen R1 coding round. The base problem is the standard first-bad-version monotonic search -- given an isBadVersion(v) oracle over versions 1..n, find the first bad version with a single binary search.

```
def first_bad_version(n, is_bad):
```

4. Find median of merged RLE arrays

MEDIUM | asked 2026-01-06 |

Overview: The question evaluates understanding of run-length encoding, order-statistics (median) computation, and efficient merging of frequency-annotated sequences, emphasizing handling large frequencies without explicit decompression.

```
def median_rle(a, b):
```

5. Solve Banana Speed and Interval Merge

MEDIUM | asked 2025-01-27 |

Constraints & Assumptions - Provide clear algorithms, correctness reasoning, complexity, and edge cases. - For minimum eating speed, each hour can be spent on only one pile. - For interval merging, intervals overlap when the next start is less than or equal to the current merged end. - Use sorted or binary-search approaches where appropriate. ### Clarifying Questions to Ask - Are piles and h always positive? - Is h at least the number of piles?

```
def min_eating_speed(piles, h):
```

6. Given a list with response time for each server, calculate how long time it will take to process n requests.

MEDIUM | asked n/a |

Given a list with response time for each server, calculate how long time it will take to process n requests.

```
def time_for(times, n, starts=None):
```

7. Find Maximum Envelope Nesting

HARD | asked 2026-03-18 |

Overview: This question evaluates algorithmic problem-solving skills, particularly reasoning about multi-dimensional ordering, sequence optimization, and time-complexity analysis.

```
def max_envelopes(envelopes):
```

8. Find the median of two sorted arrays.

HARD | asked n/a |

Find the median of two sorted arrays.

```
def kth_of_two(a, b, k):
```

9. Count of Smaller Numbers After Self

HARD | asked n/a |

Given an integer array nums, return an array counts where counts[i] is the number of smaller elements to the right of nums[i].

```
def count_smaller_after(nums):
```

10. Progressive Array Partitioning

MEDIUM | asked 2026-08-11 | array, hashmap, sorting

Examples: Stage 1 [1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true because the four value-groups have different sizes. [1, 2, 3] -> false because every value-group has size one. Stage 2 [1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true. [4, 5, 6, 7, 7, 8, 9] -> false.

```
def stage1(counts):
```

11. Streaming Points: Emit K Within Distance

MEDIUM | asked 2026-06-23 | data-structure, sorting, sliding-window, ambiguous-spec

Process a stream of float positions. The important ambiguity is whether the condition means a span of length d or pairwise distance within d.

```
class PointStream:

    def __init__(self, k, d):

    def window_start(self, pos):

    def add(self, x):
```

12. Find Containing Range

MEDIUM | asked 2026-06-04 |

Overview: This question evaluates the ability to design appropriate data structures and apply algorithmic reasoning for efficient interval containment queries, emphasizing preprocessing, correctness, and time-space trade-offs.

```
class RangeIndex:

    def __init__(self, ranges=()):

    def insert(self, s, e):

    def find(self, x):

    def find_original(self, x):

    def find_many(self, queries):
```

13. Check if all substrings are anagrams of words

MEDIUM | asked 2026-03-11 |

You are given a string s (letters only) and access to an English dictionary dict (a set of valid words). Return true if **every contiguous substring** of s with length **>= 3** can be **rearranged (anagrammed)** into a valid dictionary word. Formally, for every substring t = s[i..j] with |t| >= 3, there exists some word w in dict such that w is a permutation of t (same multiset of characters). Otherwise return false. Clarifications: - Substring means **contiguous**.

```
def all_anagram_words(s, words, min_len=3):
```

14. Find shared objects across two log files

MEDIUM | asked 2025-11-05 |

You are given two large log files representing activity on two different days. Each line of each log has three fields: - timestamp: a time value (you may treat it as an opaque string or integer) - obj_id: identifier of an object (string or integer) - client_id: identifier of a client (string or integer) The logs are not necessarily sorted by any field. Define an object as **interesting** if: 1.

```
def shared_objects(day1, day2):
```

15. Merge Intervals

MEDIUM | asked n/a |

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6].

```
def merge_intervals(intervals, touching_ints=Fa
  lse):
```

16. Group anagrams

MEDIUM | asked n/a |

Group anagrams

```
def group_by_sorting(words):
```

17. Find safe travel intervals between planet influences

HARD | asked 2025-11-22 |

You are planning a space route along a one-dimensional line (the x-axis). You are given a list of planets. Each planet is represented by an integer pair \((c, r)\): - c is the coordinate of the planet's center on the x-axis. - r is a non-negative integer radius of its gravitational influence.

```
def safe_intervals(planets):
```